Chess Playing Model using Soft Computing

Research Papers, Architecture and Flow Diagram — Explained

Subject: Soft Computing | Techniques: Genetic Algorithm + Fuzzy Logic

Part 1 — The Five Papers, Ranked by How Well They Match Our Project

The papers are ordered from the one our implementation follows most closely, down to the one that is only background reading. Knowing this order matters: if the teacher asks "how does this paper relate to your code?", you must be able to answer honestly for each one.

RANK 1 **STRONG MATCH**

Genetic Algorithms for Evolving Computer Chess Programs

David, van den Herik, Koppel & Netanyahu — *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 5, pp. 779–789, 2014 (SCIE)

What the paper does: It shows that a genetic algorithm can learn the weights of a chess evaluation function automatically, starting from completely random values. It learns first by copying grandmaster behaviour, then improves further by making the population compete against itself. The evolved program beat a two-time World Computer Chess Champion.

> **How it matches our project:** This is exactly what our Module 5 does. We encode the evaluation weights as a chromosome and evolve them with a genetic algorithm, the same core idea. This is our main reference paper and the one we cite for our method.

Journal: https://ieeexplore.ieee.org/document/6626616 **Free PDF:** https://arxiv.org/pdf/1711.08337

RANK 2 **STRONG MATCH**

An Evolutionary Algorithm with a History Mechanism for Tuning a Chess Evaluation Function

Vázquez-Fernández, Coello Coello & Sagols Troncoso — *Applied Soft Computing*, vol. 13, no. 7, pp. 3234–3247, 2013 (SCIE)

What the paper does: Instead of making candidates play full games against each other (which is very slow), it scores each candidate by showing it positions from real grandmaster games and counting how many times it picks the same move the grandmaster picked. It also keeps a record of good past candidates so they are not lost. This raised the solve rate from 40.78% to 53.08%.

> **How it matches our project:** This is our fitness function. In `train_ga.py` the setting `MODE = "positions"` implements exactly this method. It is the reason our training can finish on a laptop instead of taking days.

Journal: https://www.sciencedirect.com/science/article/abs/pii/S1568494613000690 — paywalled, use college library or ResearchGate request.

RANK 3 **PARTIAL MATCH**

History Mechanism Supported Differential Evolution for Chess Evaluation Function Tuning

Bošković, Brest, Zamuda, Greiner & Žumer — *Soft Computing* (Springer), vol. 15, no. 4, pp. 667–683, 2011 (SCIE)

What the paper does: Tunes about 190 chess evaluation parameters using Differential Evolution, a different evolutionary technique, helped by opposition-based learning and a history mechanism. Tuning improved playing strength by an average of 155.2 rating points.

> **How it matches our project:** Same problem, different algorithm. They use Differential Evolution; we use a Genetic Algorithm. So we do *not* cite this as our method. We cite it for two reasons: to show that other metaheuristics solve the same problem, and to give us a published number (155 rating points) to compare our own improvement against.

Journal: https://link.springer.com/article/10.1007/s00500-010-0593-z — paywalled.

RANK 4 PARTIAL MATCH

A Self-Learning Evolutionary Chess Program

Fogel, Hays, Hahn & Quon — *Proceedings of the IEEE*, vol. 92, no. 12, pp. 1947–1954, 2004 (SCIE)

What the paper does: Describes "Blondie25", a chess program given only the basic rules and piece values, which then improves by playing against itself. It combines an evolutionary algorithm with small neural networks that judge sections of the board. Its strength improved by nearly 400 rating points.

> **How it matches our project:** Half of it matches. The evolutionary self-play part is the same idea as our `MODE = "games"` fitness setting. The neural network part does not match, because our project deliberately uses fuzzy logic instead of neural networks. We cite it as the classic proof that evolution alone can teach a program to play chess.

Journal: https://doi.org/10.1109/JPROC.2004.837633 — paywalled.

RANK 5 BACKGROUND ONLY

A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play

Silver et al. (AlphaZero) — *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018 (SCI)

What the paper does: AlphaZero learns chess from scratch by playing millions of games against itself. It uses a deep neural network to judge positions and Monte Carlo Tree Search to choose moves. It beat Stockfish 8 with 28 wins, 72 draws and 0 losses.

> **How it matches our project: it does not.** We use no neural network, no Monte Carlo Tree Search and no reinforcement learning. Be honest about this if asked. The correct answer is that this paper is the *alternative approach we considered and rejected*, and it justifies our design choice: AlphaZero required thousands of TPUs and millions of self-play games, which is impossible on a student laptop, so we chose evolutionary tuning instead.

Journal: https://www.science.org/doi/10.1126/science.aar6404 **Free PDF:** https://arxiv.org/pdf/1712.01815

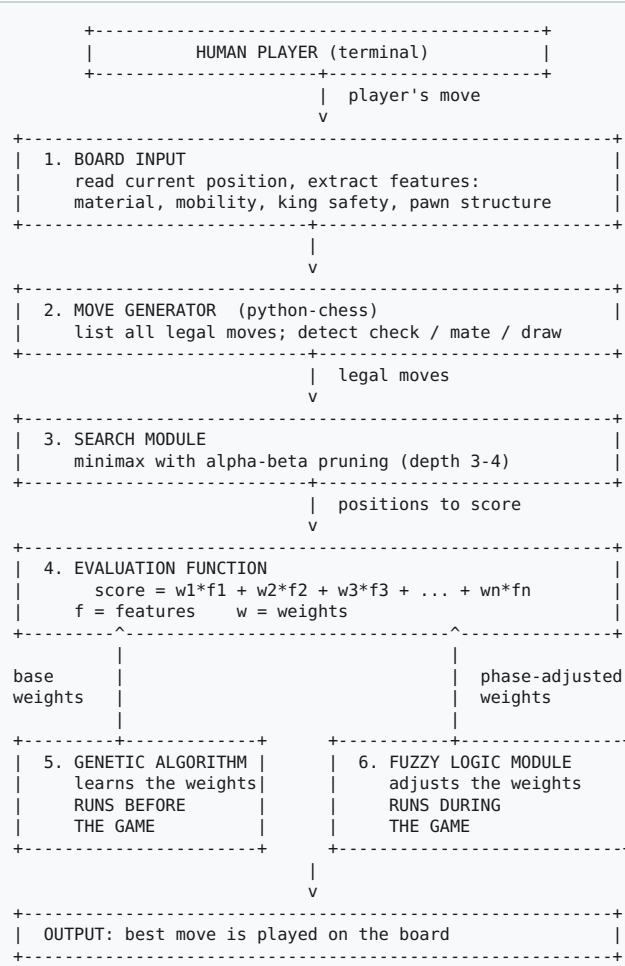
Important thing to know before your viva. None of the five papers covers the *fuzzy logic* part of our project. That is not a mistake — it is our own addition, and it is the novel part of our work. If the teacher asks which paper the fuzzy module comes from, the correct answer is:

> *"The fuzzy phase module is our own contribution. The papers we cite all tune a single fixed set of weights. We extend that idea by using fuzzy logic to shift those weights smoothly as the game moves from opening to endgame."*

Part 2 — System Architecture

Architecture and flow diagram are not the same thing.
The **system architecture** shows *what the system is made of* — the parts and how they connect. It is static, like the floor plan of a house.
The **flow diagram** shows *what happens in what order* when the program runs. It is dynamic, like the route you walk through the house.
Same system, two different views. You need both.

2.1 The architecture diagram



2.2 How to explain the architecture

Describe it in two halves.

The pipeline (Modules 1 to 4). These are stacked top to bottom and data flows straight down. The board comes in, legal moves are generated, the search looks a few moves ahead, and the evaluation function gives each position a score.

The soft-computing pair (Modules 5 and 6). These do not sit in the pipeline. They both feed into Module 4 from the side, because both of them exist only to supply the *weights* that the evaluation function uses. The key difference between them:

Module	When it runs	What it produces
5. Genetic Algorithm	Before the game (offline, once)	The base weights. Saved into `best_weights.txt` and loaded when the game starts.
6. Fuzzy Logic	During the game (live, every position)	Adjusted weights, shifted according to whether the position is opening, middlegame or endgame.

> *One sentence for the viva: "The genetic algorithm decides what the weights should be; the fuzzy module decides how those weights shift as the game goes on."*

2.3 What each module does

Module 1 — Board Input. Reads the current position and turns it into seven numbers called features: material, mobility, king safety, king activity, doubled pawns, passed pawns and centre control. A positive total means White is ahead.

Module 2 — Move Generator. Produces every legal move and detects check, checkmate, stalemate and draws. We use the python-chess library, because chess rules such as castling, en passant and promotion are complicated and rewriting them would add nothing to

the project.

Module 3 — Search. Looks ahead using minimax: we assume we play our best move, then the opponent plays their best reply, and so on. Alpha-beta pruning skips branches that cannot possibly be better, making the search much faster. We search 3 to 4 moves deep while playing, and only 1 to 2 deep during training so that it finishes quickly.

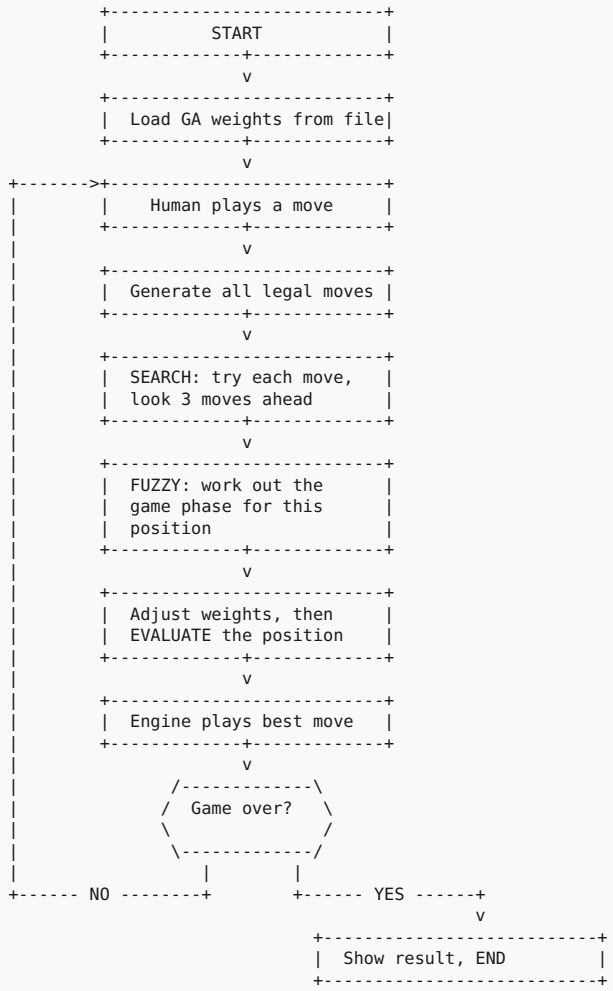
Module 4 — Evaluation Function. Turns a position into one number. It multiplies each feature by its weight and adds them all up. Everything else in the project exists to produce good weights for this one formula.

Module 5 — Genetic Algorithm. Learns those weights automatically. Explained in Part 4.

Module 6 — Fuzzy Logic. Works out how far the game has progressed and adjusts the weights to suit that stage. Explained in Part 4.

Part 3 — Flow Diagram

3.1 The flow diagram



3.2 How to explain the flow, step by step

1. **Start.** The program begins.
2. **Load GA weights.** The weights that the genetic algorithm learned earlier are read from `best_weights.txt`. Note that the genetic algorithm itself does *not* run here — it already finished before the game started.
3. **Human plays a move.** The user types a move such as `e2e4`. It is checked for legality.
4. **Generate legal moves.** The engine lists every legal reply available to it.
5. **Search.** For each of those replies the engine looks a few moves ahead, building a tree of possible futures.
6. **Fuzzy phase.** At the end of every line searched, the fuzzy module looks at how much material is left and what move number it is, and works out how much the position belongs to the opening, the middlegame and the endgame.
7. **Adjust and evaluate.** Those phase values reshape the weights, and the evaluation function then scores the position using the adjusted weights.
8. **Engine plays best move.** The move leading to the best score is chosen and played.
9. **Game over?** If not, the loop returns to step 3 and the human moves again. If yes, the result is displayed and the program ends.

The two points examiners usually ask about.
1. Where is the genetic algorithm in this flow? It is not in the loop. It runs once, beforehand, as a separate training program. Only its *output* (the weights file) enters the flow, at step 2.
2. Where is the fuzzy logic? Inside the loop, at step 6 — it runs for every single position the search examines, which can be thousands of times per move.

Part 4 — The Two Concepts Explained

4.1 Genetic Algorithm

A genetic algorithm is a search method copied from evolution in nature. You use it when you need to find good numbers but have no idea what those numbers should be.

Our evaluation function needs seven weights. Nobody knows the correct values. So instead of guessing, we make many random guesses, score them, keep the good ones, mix them together, add small random changes, and repeat. The guesses improve every round.

Term	Meaning in our project
Chromosome	One complete set of seven weights. One candidate answer.
Gene	One single weight inside that set.
Population	All the chromosomes in the current round. We use 12.
Fitness	The score for a chromosome — how well it plays, or how many grandmaster moves it guesses correctly.
Selection	Choosing the higher-scoring chromosomes to become parents. We use tournament selection: pick 3 at random, keep the best.
Crossover	Mixing two parents to produce a child set of weights.
Mutation	A small random change to a weight, so that new ideas can appear.
Generation	One complete round of scoring, selecting, crossing over and mutating. We run 6.
Elitism	Copying the best 2 chromosomes unchanged into the next generation, so the best answer is never lost.

> *One sentence: "A genetic algorithm searches for good numbers by keeping what works, mixing it, and mutating it — the same way breeding works in nature."*

4.2 Fuzzy Logic

Ordinary computer logic is either / or. Something is true or false, 1 or 0. But many real ideas have no sharp boundary. There is no exact moment when a chess game becomes an endgame.

Fuzzy logic allows something to be **partly true**. A position can be 30% middlegame and 70% endgame at the same time. This lets the weights change smoothly instead of jumping suddenly.

Step	What happens in our project
Crisp input	Two ordinary numbers: material left on the board, and the move number.
Membership function	A simple graph (ours are triangles) that says how much a number belongs to a word. Material 24 is partly LOW and partly MEDIUM.
Fuzzification	Turning the crisp numbers into those fuzzy words: material becomes LOW / MEDIUM / HIGH, move number becomes EARLY / MIDDLE / LATE.
Fuzzy rules	Plain IF-THEN statements. IF material is HIGH AND moves are EARLY THEN opening. IF material is LOW THEN endgame.
Inference	Applying all the rules at once. We use the Mamdani method, where AND is calculated as the minimum of the two values.
Defuzzification	Turning the fuzzy result back into usable numbers — three phase values that add up to 1.0.

Actual output from our program:

Material left	Move number	Opening	Middlegame	Endgame
78 (start)	1	1.00	0.00	0.00
52	14	0.30	0.70	0.00
40	25	0.00	1.00	0.00
24	35	0.00	0.42	0.58
12	45	0.00	0.00	1.00

Notice the middle rows. The position is not forced to be one phase or another — it blends. That gradual change is what fuzzy logic gives us, and a plain IF statement could not.

> *One sentence: "Fuzzy logic handles ideas that have no sharp boundary, by allowing partial membership instead of a yes/no answer."*

4.3 Why the two work together

A single fixed set of weights is not ideal, because good chess changes as the game goes on. In the middlegame the king must stay safe behind its pawns. In the endgame that same king should march into the centre and become active. The genetic algorithm alone cannot capture this, because it produces one set of weights for the whole game.

So the fuzzy module multiplies the learned weights by phase-dependent amounts. For example the king-safety weight is multiplied by 1.4 in the middlegame but only 0.2 in the endgame, while king activity goes from 0.0 in the opening up to 1.5 in the endgame. Because the phase values blend smoothly, the weights slide gradually rather than jumping.